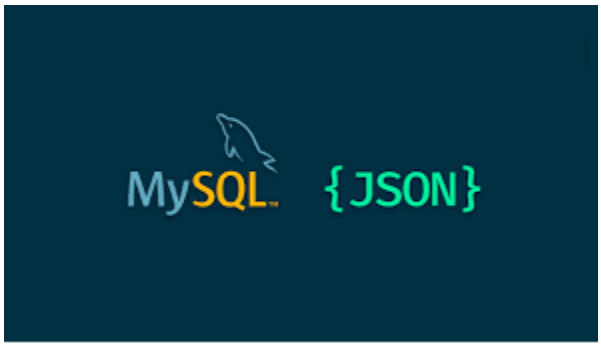


1 Introducción 2
2 Creación de una tabla con columnas JSON 3
3 Insertar datos en columnas JSON 4
4 Consultas básicas con JSON 5
5 Modificación de datos JSON 6
5.1 Modificar contenidos de un array 7
6 Índices y optimización para columnas JSON 8

1 Introducción

En las bases de datos relacionales tradicionales, como *MySQL*, las tablas están estructuradas tradicionalmente con un **esquema fijo**, es decir, cada fila tiene un conjunto definido de columnas con tipos de datos preestablecidos. Esta estructura relacional funciona muy bien para almacenar datos altamente estructurados y con relaciones bien definidas entre las tablas. Sin embargo, hay escenarios donde los datos no encajan tan fácilmente en este modelo rígido y estructurado. Aquí es donde entra en juego el uso de ***JSON (JavaScript Object Notation)***.

Actualmente, los sistemas de bases de datos relacionales más potentes permiten trabajar con contenidos *JSON*, lo que facilita el trabajo de cierto tipo de aplicaciones que requieren esquemas más flexibles y demandan trabajar con datos semi-estructurados o no estructurados

A lo largo de los años, las bases de datos relacionales han evolucionado para integrar características que anteriormente eran exclusivas de las bases de datos *NoSQL*. Esto incluye el soporte para tipos de datos como *JSON*, lo que permite combinar lo mejor de ambos mundos:

- **Estructura y consistencia transaccional** de las bases de datos relacionales.
- **Flexibilidad** en el manejo de datos semi-estructurados o no estructurados, característica común de las bases de datos *NoSQL*.

Esto hace que *MySQL*, y otras bases de datos relacionales, sean una opción viable para aplicaciones que necesitan, tanto la robustez de las bases de datos tradicionales, como la flexibilidad de los formatos más modernos.

A pesar de que los modelos relacionales permitan hoy en día trabajar sobre datos *JSON*, los sistemas ***NoSQL*** como ***MongoDB*** tienen ventajas claras dependiendo del caso de uso:

- **Escalabilidad del sistema**. Las herramientas *NoSQL* suelen estar pensadas para escalado horizontal, lo que las hace más adecuadas para entornos de *Big Data* y para entornos replicados.
- **Rendimiento en ciertos escenarios**. *MongoDB* suele ser más rápido, ya que se ha diseñado desde el principio con esa finalidad.
- **Permite mayor flexibilidad**. Incorpora funciones y herramientas adicionales que facilitan el trabajo con este tipo de contenidos.

MySQL* proporciona una serie de funciones y operadores específicos para trabajar con datos *JSON, lo que permite a los desarrolladores realizar consultas complejas directamente sobre los datos almacenados en formato *JSON*. Esto incluye la capacidad de extraer valores de un objeto *JSON*, filtrar registros según las propiedades de un *JSON* y realizar modificaciones dentro de los datos.

2 Creación de una tabla con columnas JSON

Desde la ***versión 5.7***, ***MySQL*** **introdujo el tipo de dato *JSON***, que podemos usar como un tipo más en nuestras tablas.

Por ejemplo:

```
CREATE TABLE productos (
   id INT AUTO_INCREMENT PRIMARY KEY,   -- Campo para el identificador único del producto
   nombre VARCHAR(255) NOT NULL,        -- Campo para el nombre del producto
   detalles JSON                        -- Columna para almacenar detalles en formato JSON
);
```

A diferencia de otros tipos de datos como *TEXT* o *VARCHAR*, el tipo *JSON* permite almacenar y manipular estructuras complejas como objetos y arrays dentro de la base de datos.

Una vez que la tabla ha sido creada, el siguiente paso es insertar datos en ella. Dado que el campo *detalles* es de tipo *JSON*, puedes almacenar cualquier estructura *JSON* válida dentro de esa columna. Por ejemplo:

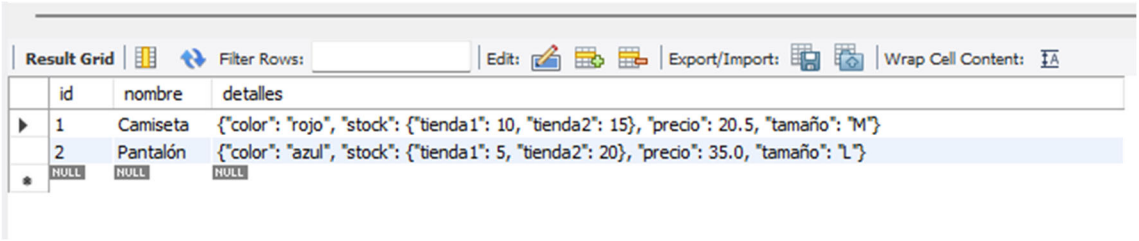
```
INSERT INTO productos (nombre, detalles) VALUES
('Camiseta', '{"color": "rojo", "tamaño": "M", "precio": 20.5, "stock": {"tienda1": 10, "tienda2": 15}}'),
('Pantalón', '{"color": "azul", "tamaño": "L", "precio": 35.0, "stock": {"tienda1": 5, "tienda2": 20}}');
```

MySQL **valida automáticamente si los datos insertados en un campo de tipo *JSON* son válidos**. Si intentáramos insertar datos que no son válidos, *MySQL* devolvería un error.

Una vez que tenemos datos almacenados en una columna de tipo *JSON*, podemos realizar consultas para acceder a ellos. Por ejemplo, el clásico:

```
SELECT * FROM productos;
```

Devolvería:



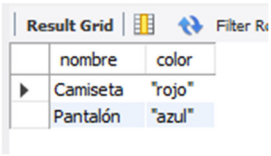
Si queremos acceder a un valor específico dentro del JSON, podemos usar la función ***JSON_EXTRACT*** o el operador ***->*****,** a la que le podríamos indicar la **clave específica** dentro del JSON. Por ejemplo:

```
SELECT nombre, JSON_EXTRACT(detalles, '$.color') AS color
FROM productos;
```

O

```
SELECT nombre, detalles->'$.color' AS color
FROM productos;
```

Devolverían:



A la hora de especificar la clave, El símbolo ***$.*** indica la raíz del JSON.

3 Insertar datos en columnas JSON

Hemos visto en el punto anterior, que es posible insertar datos de tipo *JSON* en una tabla usando la sintaxis estándar de ***INSERT INTO***. Lo único que tenemos que tener en cuenta es que los valores asociados a campos de tipo JSON deben ser cadenas JSON válidas. Recordemos que un campo JSON puede usar diferentes tipos de datos asociados a sus distintas claves:

- **Cadenas de texto** (string): *"color": "rojo"*
- **Números** (integer o decimal): *"precio": 19.99*
- **Booleanos** (true o false): *"disponible": true*
- **Arreglos** (arrays): *"colores": ["rojo", "azul", "verde"]*
- **Objetos anidados** (objetos JSON): *"stock": {"tienda1": 20, "tienda2": 50}*
- **Nulos** (null): *"descuento": null*

Es posible almacenar *arrays* y *objetos* asociados a los campos *JSON*, lo cual es muy útil cuando tenemos datos que tienen múltiples valores o estructuras complejas. Por ejemplo:

```
INSERT INTO productos (nombre, detalles)
VALUES ('Camiseta', '{
   "color": "rojo",
   "tamaño": "L",
   "precio": 20.5,
   "colores_disponibles": ["rojo", "azul", "verde"],
   "stock": {"tienda1": 10, "tienda2": 25},
   "descuento": null
}');
```

Recalquemos nuevamente el hecho de que **el propio *MySQL* se encarga de validar el formato de los datos de tipo *JSON* que queremos insertar** y que, si tuviera algún fallo, la operación no se llevaría a cabo.

4 Consultas básicas con JSON

Una vez que hemos almacenado datos en una columna *JSON*, necesitamos aprender a consultarlos. MySQL ofrece varias funciones y operadores para acceder a valores dentro de un objeto *JSON* y realizar consultas avanzadas.

Para acceder a valores individuales dentro de una columna *JSON*, *MySQL* proporciona tres formas principales:

- ***JSON_EXTRACT()*** → Función para extraer valores de *JSON*.
- **Operador ->** (equivalente a *JSON_EXTRACT*). Devuelve los valores en **formato *JSON*.**
- **Operador ->>** Para obtener valores como **texto sin formato**.

Podemos utilizar los operadores y la función anterior, en las diferentes cláusulas de la sentencia SQL. Por ejemplo:

```
SELECT * FROM productos
WHERE detalles->'$.color' = 'rojo';
```

```
SELECT nombre, detalles->>'$.stock.tienda1' AS stock_tienda1
FROM productos
WHERE detalles->>'$.stock.tienda1' > 15;
```

```
SELECT nombre, detalles->>'$.precio' AS precio
FROM productos
ORDER BY detalles->>'$.precio' ASC;
```

```
SELECT nombre, detalles->>'$.tamaño' AS tamaño
FROM productos
WHERE detalles->>'$.tamaño' LIKE 'M';
```

Si en *SELECT* hiciéramos referencia a un campo *JSON* que solo existe para alguno de los registros, devolvería **null** en aquellos para los que no existe.

Si el JSON contiene un array en lugar de un objeto, podemos acceder a sus elementos mediante índices []. Por ejemplo:

```
SELECT JSON_EXTRACT(detalles, '$.tallas[0]') AS primera_talla FROM productos;
```

Si no sé la posición que ocupa el elemento en el array y quisiera saber, por ejemplo, si alguna de las tallas del array es la 'M', podría usar la función ***JSON_CONTAINS()***.

```
SELECT * FROM productos
WHERE JSON_CONTAINS(detalles->'$.tallas', 'M');
```

Si lo que quisiéramos es buscar, si al menos un valor de un conjunto existe en el array, podríamos usar la función ***JSON_OVERLAPS()***.

```
SELECT * FROM productos
WHERE JSON_OVERLAPS(detalles->'$.tallas', '["S", "M", "XL"]');
```

5 Modificación de datos JSON

MySQL permite modificar los valores dentro de una columna *JSON* sin tener que sobrescribir el objeto completo (que también podría hacerse). Para ello, podemos usar funciones específicas según el tipo de modificación que necesitemos:

- ***JSON_SET(json_doc, path, valor, ...)*** Añade una nueva clave o actualiza una existente. Si la clave existe la actualiza y si no, la crea.

  ```
  UPDATE productos
  SET detalles = JSON_SET(detalles, '$.precio', 25.0)
  WHERE nombre = 'Camiseta';
  ```

- ***JSON_REPLACE(json_doc, path, valor, ...)*** Reemplaza solo si la clave ya existe. Si no existe, no hace nada.

  ```
  UPDATE productos
  SET detalles = JSON_REPLACE(detalles, '$.stock', 20)
  WHERE nombre = 'Camiseta'
  ```

- ***JSON_INSERT(json_doc, path, valor, ...)*** Inserta solo si la clave no existe. Si la clave ya existe, no hace nada

  ```
  UPDATE productos
  SET detalles = JSON_INSERT(detalles, '$.talla', 'M')
  WHERE nombre = 'Camiseta';
  ```

También, es posible usar múltiples pares en una sola llamada:

```
UPDATE productos
SET detalles = JSON_SET(detalles,
   '$.precio', 29.99,
   '$.color', 'verde',
   '$.disponible', true
)
WHERE nombre = 'Camiseta';
```

Si lo que queremos es **eliminar una clave**, debemos usar la función ***JSON_REMOVE()*:**

```
UPDATE productos
SET detalles = JSON_REMOVE(detalles, '$.stock')
WHERE nombre = 'Camiseta';
```

5.1 Modificar contenidos de un array

Trabajar con arrays dentro de columnas *JSON* en *MySQL* es muy útil, pero también requiere algunos truquitos porque no hay funciones directas para modificar un array como si fuera una lista en programación tradicional. Sin embargo, sí podemos hacerlo manipulando el array por su índice o reconstruyéndolo parcialmente.

Supongamos que tenemos este producto:

```
INSERT INTO productos (nombre, detalles)
VALUES ('Zapatillas', '{"tallas": ["38", "39", "40", "41"]}');
```

Vamos a ver cómo hacer diferentes tipos de acciones sobre él:

- **Modificar un elemento por el índice**. Si sabemos el índice, resulta sencillo:

```
UPDATE productos
SET detalles = JSON_SET(detalles, '$.tallas[1]', '39.5')
WHERE nombre = 'Zapatillas';
```

- **Modificar un elemento de un array sin conocer su índice**. *MySQL* no tiene una función para hacer esto directamente y el proceso es un poco más laborioso. Podemos hacerlo haciendo uso de las funciones ***JSON_UNQUOTE()*** y ***JSON_SEARCH()***. La primera quita las comillas de una ruta *JSON* y la segunda encuentra la ruta al valor que queremos modificar. El parámetro 'one' indica que te devolverá la primera coincidencia. Por ejemplo, si quisiéramos sustituir en el array de *colores_disponibles* el valor *'verde'* por *'amarillo'* en el producto con *id=4*:

```
UPDATE productos
SET detalles = JSON_REPLACE(
  detalles,
  JSON_UNQUOTE(JSON_SEARCH(detalles, 'one', 'verde', NULL, '$.colores_disponibles')),
  'amarillo'
)
WHERE id = 4;
```

- **Eliminar un elemento de un array si sabemos su posición**. Sabiendo su índice es más fácil:

```
UPDATE productos
SET detalles = JSON_REMOVE(detalles, '$.tallas[2]')
WHERE nombre = 'Zapatillas';
```

- **Eliminar un elemento de un array si no sabemos su posición**. Podemos proceder de forma análoga a como hemos hecho previamente con ***JSON_UNQUOTE()*** y ***JSON_SEARCH():***

```
UPDATE productos
SET detalles = JSON_REMOVE(
  detalles,
  JSON_UNQUOTE(JSON_SEARCH(detalles, 'one', 'verde', NULL, '$.colores_disponibles'))
)
WHERE id = 4;
```

- **Insertar un nuevo valor en el array**. Lo normal es añadirlo al final del array con la función ***JSON_ARRAY_APPEND()***. Si quisiéramos insertarlo en una posición determinada tendríamos que reconstruir el array nuevamente. Por ejemplo:

```
UPDATE productos
SET detalles = JSON_SET(
  detalles,
  '$.tallas',
  JSON_ARRAY_APPEND(detalles->'$.tallas', '$', '42')
)
WHERE nombre = 'Zapatillas';
```

6 Índices y optimización para columnas JSON

Las columnas JSON ofrecen mucha flexibilidad, pero **no están diseñadas originalmente para búsquedas rápidas**, como lo estarían columnas tradicionales (como *VARCHAR*, *INT*, etc.). Sin embargo, ***MySQL* te permite crear índices virtuales o generados** para mejorar el rendimiento al consultar datos específicos dentro de un campo *JSON*.

Para hacerlo, hay que crear lo que se llama una ***'columna generada'*** sobre un campo del *JSON* y posteriormente, **crear un índice sobre ese campo**. Por ejemplo:

```
ALTER TABLE productos
ADD color_virtual VARCHAR(50)
GENERATED ALWAYS AS (JSON_UNQUOTE(JSON_EXTRACT(detalles, '$.color')))
STORED;

CREATE INDEX idx_color_virtual ON productos(color_virtual);
```

Sabemos que cuando trabajamos con columnas *JSON*, no todos los registros tienen siempre la misma estructura y puede ocurrir que el campo sobre el que creamos el índice, no esté en todos los registros. **MySQL no se va a quejar** y simplemente asignará NULL a los registros donde esa clave no exista, permitiendo construir el índice.